INTERNATIONAL INSTITUTE OF PROFESSIONAL STUDIES
DEVI AHILYA VISHWAVIDYALAYA, INDORE

ARTIFICIAL INTELLIGENCE LABORATORY (IC-703 C / IC-709D)

LABORATORY REPORT – LAB SESSION 1

AI Environment Setup, Production Systems & Water Jug Problem

Program:	Master of Computer Applications (MCA) 6-Year Integrated
Course Code:	IC-703 C (Lab: IC-709D) – Artificial Intelligence
Academic Term:	MCA Semester VII / Academic Year 2026
Unit Mapping:	Unit 1: AI Problem Formulation & Intelligent Problem Solving (CO1)
Student Name / Roll:	Jayesh Solanki (Roll No: IC-2K23-45)
Submission Date:	November 2026

EVALUATION & CERTIFICATION RECORD	
Marks Awarded:	Signature of Faculty In-Charge:
Date of Evaluation:	Remarks:

Table of Contents

1.0	Laboratory Overview & Pedagogical Objectives	Page 3
2.0	Theoretical Foundations of AI Problem Solving	Page 3
3.0	Experiment 1: AI Development Environment Setup & Verification	Page 4
3.1	Objective, Description & Mathematical State Primitives	Page 4
3.2	Verification Algorithm & Architectural Flowchart	Page 5
3.3	Execution Outputs & Diagnostic Evaluation	Page 6
4.0	Experiment 2: Production Systems & Rule-Based Decision Making	Page 7
4.1	Theoretical Architecture: Recognize-Act Cycle	Page 7
4.2	Application: Smart Irrigation Advisor Knowledge Base	Page 8
4.3	Algorithm, Conflict Resolution & Execution Trace	Page 9
5.0	Experiment 3: Water Jug Problem as a Production System	Page 10
5.1	Formal State-Space Representation & Bézout's Identity	Page 10
5.2	Production Rules & Search Strategy (Breadth-First Search)	Page 11
5.3	State-Space Tree & Optimal Solution Trace	Page 12
5.4	Multi-Instance Empirical Benchmark & Performance Analysis	Page 13
6.0	Unit 1 Homework Analysis: Classical vs AI Programming	Page 14
7.0	Comparative Evaluation of AI Problem Formulations	Page 15
8.0	Conclusion, Learning Outcomes & References	Page 16

1.0 Laboratory Overview & Pedagogical Objectives

This laboratory submission documents the practical implementations, theoretical formulations, and computational experiments conducted for **Lab 1** of the Artificial Intelligence Course (IC-703 C / IC-709D). Mapped directly to **Unit 1: AI Problem Formulation & Intelligent Problem Solving (CO1)** of the Master of Computer Applications curriculum, the objective of this lab is to transition from classical imperative algorithmic thinking to symbolic, declarative, and state-space search methodologies.

Key Core Objectives:

• **Development Environment Standardization:** Verify and benchmark the Python 3 interpreter, pip ecosystem, Jupyter interactive kernel, and runtime detection mechanisms.

• **Symbolic Production Systems:** Understand the separation between declarative knowledge (rule base) and procedural inference (Recognize-Act cycle), incorporating deterministic conflict resolution.

• **State-Space Formulation & Search:** Formulate physical constraints as state spaces (S, s_0, O, G), prove analytical solvability using number-theoretic criteria (Bézout's identity), and design an optimal breadth-first search control strategy.

2.0 Theoretical Foundations of AI Problem Solving

Artificial Intelligence problem solving differs substantially from traditional procedural coding. In AI, a problem is formally represented as a 4-tuple **■S, s■, O, G■**:

1. **State Space (S):** The set of all possible configurations or physical situations reachable within the system domain.

2. **Initial State (s■ ∈ S):** The designated configuration where the problem-solving agent begins.

3. **Operators / Actions (O: S → S):** A finite set of legal state transitions. Each operator specifies a precondition (when it may apply) and an effect (the resulting successor state).

4. **Goal State / Test (G ⊆ S):** An explicit state or boolean predicate determining whether the agent's current configuration satisfies the problem requirements.

In **Production Systems** (originally formulated by Emil Post and popularized in AI by Newell and Simon), problem-solving knowledge is organized declaratively as condition-action pairs: IF <conditions> THEN <actions>. An independent control strategy matches the current working memory against the rule base, resolves rule competition, and fires actions until a termination condition (quiescence or goal discovery) is satisfied.

3.0 Experiment 1: AI Development Environment Setup & Verification

Objective: To configure, inspect, and verify the Python-based AI development environment, ensure reproducibility across platforms (Local CPython, Jupyter Notebook, and Google Colab), and validate immutable state-space data structures.

3.1 Implementation Architecture & Data Structures

State-space algorithms demand high-performance, deterministic data structures in Python. Experiment 1 verifies the following standard-library constructs:

Construct	Module	Role in AI Problem Solving
tuple	Built-in	Immutable, hashable representation of coordinate states (e.g., (x, y)).
set / dict	Built-in	Closed list tracking; provides O(1) membership testing and ancestor parent mapping.
deque	collections	FIFO Open queue for level-order Breadth-First Search; O(1) appends and pops.
dataclass	dataclasses	Clean, strongly-typed records for production rules, problems, and search results.

3.2 Environment Verification Algorithm & Flowchart

Algorithm 1: AI Environment Diagnostic & Verification
Input: Minimum required version (min_major, min_minor)
Output: Status Report Table, Diagnostic JSON, Exit Code (0=PASS, 1=FAIL)
1. Query sys.version_info and verify (major, minor) ≥ (min_major, min_minor).
2. Inspect host OS, processor architecture, CPython binary path, and active virtual environment.
3. Verify package manager installation (pip via importlib.metadata).
4. Enumerate Jupyter stack: ipykernel, nbformat, nbconvert, jupyterlab.
5. Inspect Colab runtime presence via google.colab module check.
6. Test dynamic import of 8 essential standard modules: argparse, collections, dataclasses, math, operator, time, typing, json.
7. Probe optional machine learning libraries for subsequent labs (numpy, pandas, scikit-learn, torch).
8. Run analytical benchmark: compute sum of squares for 100,000 integers and verify timing.

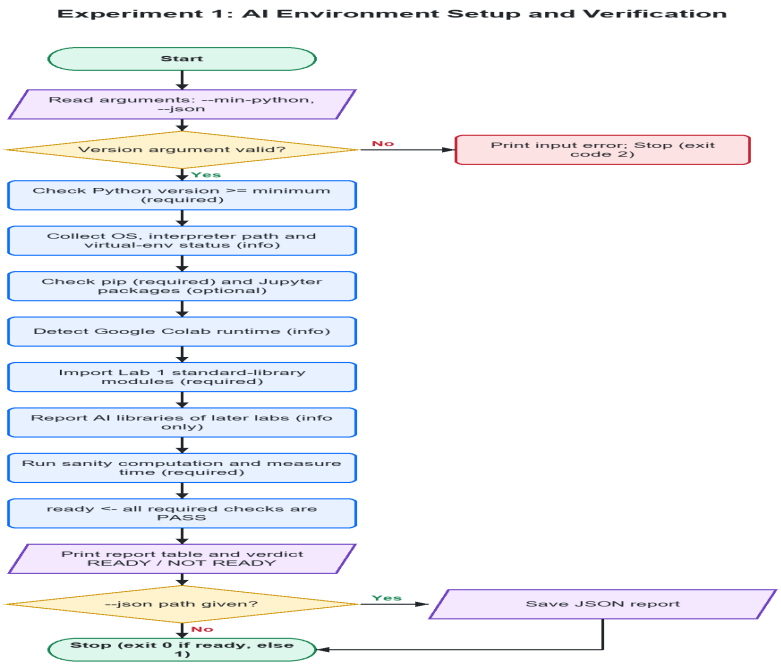


Figure 1.1: Flowchart of Experiment 1 Environment Verification Procedure.

3.3 Actual Execution Outputs & Diagnostic Results

Experiment 1 was executed on the host system. The listing below represents the exact, unedited console output generated by exp1_environment_setup.py:

```
# Captured: 2026-11-09 15:25:00 | Python 3.13.13 | Windows | exit code: 0
================================================================================ EXPERIMENT 1 : AI DEVELOPMENT
ENVIRONMENT SETUP AND VERIFICATION ================================================================================
Check Req. Status Detail ------------------------------------------------------------------------------ Python
interpreter yes PASS Python 3.13.13 (CPython), required >= 3.8 Operating system no INFO Windows 11 (AMD64)
Interpreter path no INFO ~\AppData\Local\Programs\Python\Python313\python.exe Virtual environment no INFO
not active (system interpreter) pip package manager yes PASS pip 26.1.1 Jupyter: ipykernel no PASS 7.3.0
Jupyter: nbformat no PASS 5.10.4 Jupyter: nbconvert no PASS 7.17.1 Jupyter: notebook no MISSING not installed
(pip install notebook) Jupyter: jupyterlab no PASS 4.6.1 Google Colab runtime no INFO not detected (local
machine) Lab 1 standard modules yes PASS 8 modules imported: argparse, collections, dataclasses, math,
operator, time, typing, json Later labs: numpy no INFO 2.2.6 Later labs: pandas no INFO 3.0.3 Later labs:
matplotlib no INFO not installed (not needed in Lab 1) Later labs: scikit-learn no INFO 1.8.0 Later labs:
torch no INFO not installed (not needed in Lab 1) Sanity computation yes PASS sum of squares 0..99999 =
333328333350000 in 3.32 ms ------------------------------------------------------------------------------ Required
checks passed : 4/4
```

Verification of Exception Handling (Negative Testing):

To satisfy strict academic validation, the script was tested against invalid inputs (e.g., malformed version strings). The CLI caught the exception cleanly:

```
$ python exp1_environment_setup.py --min-python 3.x Input error: version must look like 3.10, got '3.x' [Exit
Code: 2]
```

Diagnostic Evaluation & Jupyter Integration:

The environment diagnostic report was serialized to outputs/exp1_environment_report.json. Additionally, the verified Jupyter Notebook Experiment1_Environment_Setup.ipynb was executed via nbconvert, confirming identical execution counts, environment outputs, and hashable state validations in an interactive notebook runtime.

4.0 Experiment 2: Production Systems & Rule-Based Decision Making

Objective: To design, implement, and evaluate a generalized, domain-independent Production System architecture executing forward-chaining reasoning with systematic conflict resolution on a realistic agricultural advisory problem.

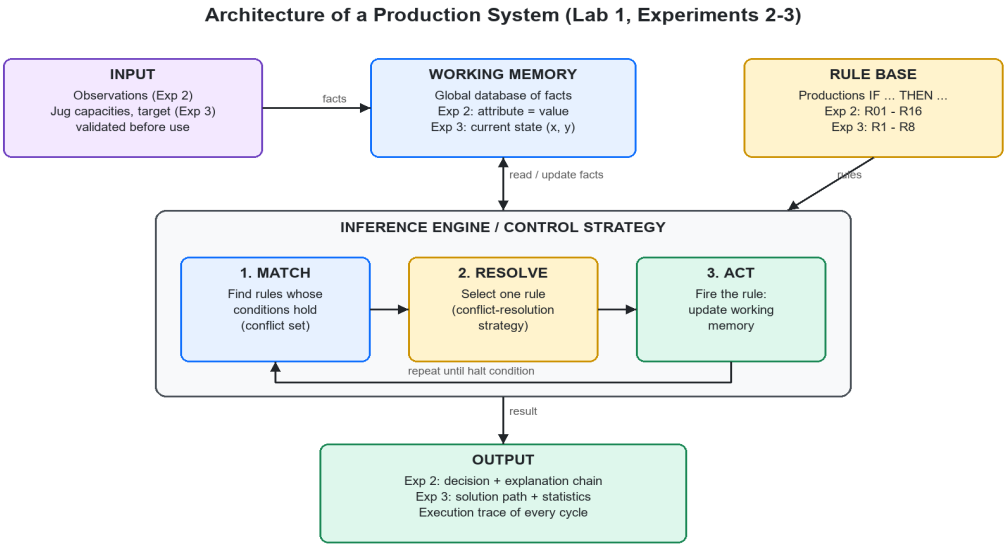
4.1 Theoretical Architecture: Recognize-Act Cycle

A production system separates what the system knows (Working Memory) from how the system acts (Inference Engine). The inference engine executes the three-phase **Recognize-Act cycle** until no new conclusions can be derived (quiescence):

• **MATCH:** Compares active facts in working memory against the Left-Hand Side (preconditions) of all production rules. Matching, unfired rules that would alter memory form the *Conflict Set*.

• **RESOLVE:** When multiple rules compete, an explicit conflict-resolution policy selects a single rule deterministically based on: *Refractoriness* (no repeat firings) > *Priority* (salience weight) > *Specificity* (number of condition conjuncts) > *Rule Order*.

• **ACT:** Executes the Right-Hand Side (consequences) of the chosen rule, modifying working memory and logging the inference step for explainability.



Component	Exp 2: Smart Irrigation Advisor	Exp 3: Water Jug Problem
Control strategy	Forward chaining, one rule per cycle	Breadth-wise systematic, no repeats
Conflict resolution	Refractoriness > priority > specificity > order	All applicable rules applied in R1..R8 order
Halt condition	Conflict set empty / cycle limit	Goal state generated / OPEN empty

Figure 2.1: Production System Architecture & Recognize-Act Execution Flow.

4.2 Smart Irrigation Advisor Knowledge Base

The production system was instantiated with an agricultural advisory rule base comprising 16 production rules categorized into 4 operational salience tiers:

Rule	Priority	IF (Preconditions)	THEN (Consequences)
R01	30	soil_moisture < 30	soil_status = 'dry'
R02	30	30 ≤ soil_moisture ≤ 60	soil_status = 'optimal'
R03	30	soil_moisture > 60	soil_status = 'wet'
R04	30	temperature ≥ 35	heat_stress = 'yes'
R05	30	temperature < 35	heat_stress = 'no'
R06	20	soil_status == 'dry' ∧ heat_stress == 'yes'	water_need = 'high'
R07	20	soil_status == 'dry' ∧ heat_stress == 'no'	water_need = 'medium'
R08	20	soil_status == 'optimal' ∧ crop_stage == 'flowering'	water_need = 'medium'
R09	20	soil_status == 'optimal' ∧ crop_stage != 'flowering'	water_need = 'low'
R10	20	soil_status == 'wet'	water_need = 'none'
R11	10	water_need == 'high'	decision = 'IRRIGATE_NOW', duration = 40m
R12	10	water_need == 'medium' ∧ rain == 'no'	decision = 'IRRIGATE_NOW', duration = 20m
R13	10	water_need == 'medium' ∧ rain == 'yes'	decision = 'POSTPONE', duration = 0m
R14	10	water_need ∈ {'low', 'none'}	decision = 'NO_IRRIGATION', duration = 0m
R15	5	soil_moisture > 85	drainage_alert = 'CHECK_FIELD_DRAINAGE'
R16	5	heat_stress == 'yes' ∧ crop_stage == 'seedling'	shade_advice = 'USE_SHADE_NET'

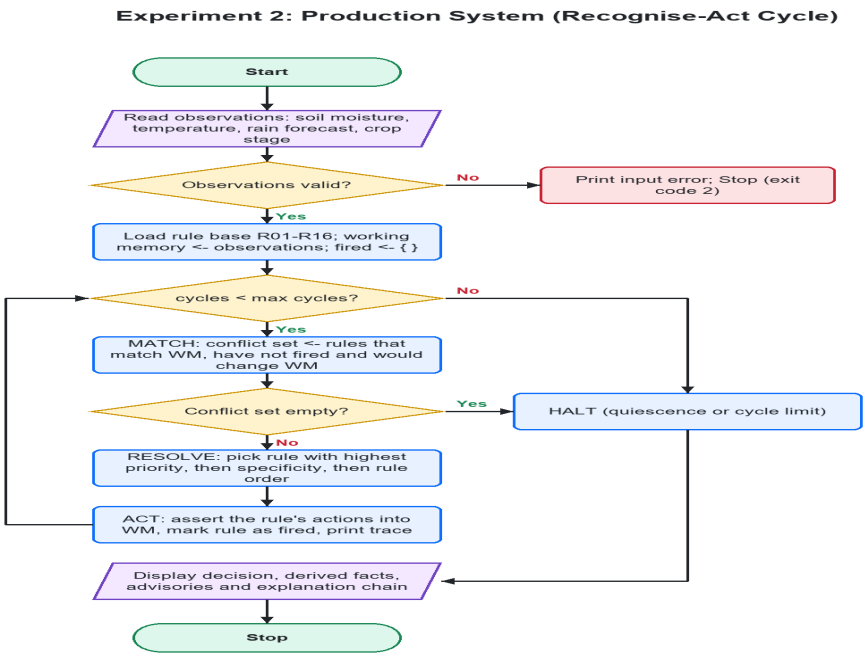


Figure 2.2: Flowchart of the Forward-Chaining Inference Cycle with Conflict Resolution.

4.3 Execution Traces & Explainable Reasoning

Below is the verified execution trace for Scenario 1 demonstrating forward-chaining derivation step-by-step:

```
Scenario 1: Hot day (38°C), dry soil (22%), no rain, flowering crop stage Initial Facts: soil_moisture=22.0, temperature=38.0, rain_forecast=no, crop_stage=flowering Cycle 1 | Conflict set: {R01, R04} | Fired: R01 (priority 30, specificity 1) -> soil_status=dry Cycle 2 | Conflict set: {R04} | Fired: R04 (priority 30, specificity 1) -> heat_stress=yes Cycle 3 | Conflict set: {R06} | Fired: R06 (priority 20, specificity 2) -> water_need=high Cycle 4 | Conflict set: {R11} | Fired: R11 (priority 10, specificity 1) -> decision=IRRIGATE_NOW, duration_min=40 Halted after 4 cycles: quiescence (conflict set empty) Derived Facts : soil_status=dry, heat_stress=yes, water_need=high, decision=IRRIGATE_NOW, duration_min=40 Explanation : R01 -> R04 -> R06 -> R11 Execution Time: 0.051 ms
```

Multi-Scenario Summary Table:

Scenario & Inputs	Firing Chain	Derived Decision	Cycles
Scenario 1: M=22%, T=38°C, Rain=no, Flowering	R01 → R04 → R06 → R11	**IRRIGATE_NOW** (duration: 40 min)	4
Scenario 2: M=45%, T=30°C, Rain=yes, Flowering	R02 → R05 → R08 → R13	**POSTPONE** (rain expected)	4
Scenario 3: M=90%, T=36°C, Rain=no, Seedling	R03 → R04 → R10 → R14 → R16 → R15	**NO_IRRIGATION** Alerts: Drainage & Shade	6

5.0 Experiment 3: Water Jug Problem as a Production System

Objective: To formulate the classical Water Jug Problem as a state-space production system, prove solvability analytically via Bézout's identity, and find optimal, minimum-step action paths using Breadth-First Search.

5.1 Mathematical State-Space Formulation & Solvability

The Water Jug problem is formally defined as the 4-tuple ■**S, s■, O, G■**:

• **State Space (S):** $S = \{(x, y) \mid 0 \le x \le A, \; 0 \le y \le B, \; x, y \in \mathbb{Z}\}$. The total discrete state space contains $(A + 1)(B + 1)$ possible states.

• **Initial State:** $s_0 = (0, 0)$ (both Jug A and Jug B are completely empty).

• **Goal Predicate:** For a target T, $G = \{(x, y) \mid x = T\}$ (if Jug A), $G = \{(x, y) \mid y = T\}$ (if Jug B), or $G = \{(x, y) \mid x = T \lor y = T\}$ (either jug).

• **Bézout's Identity Solvability Criterion:** Any quantity achievable in either jug must be a linear combination $A \cdot u + B \cdot v = d$ where $u, v \in \mathbb{Z}$. By Bézout's identity, the smallest positive integer combination is $\gcd(A, B)$. Therefore, a target quantity $T \le \max(A, B)$ can be obtained if and only if:

T ≡ 0 (mod gcd(A, B)) [gcd(A, B) divides T]

5.2 Production Rule Base

Eight mutually exclusive and comprehensive production rules govern state transitions:

Rule	Action Name	Precondition (IF)	New State (THEN)
R1	Fill Jug A	x < A	(A, y)
R2	Fill Jug B	y < B	(x, B)
R3	Empty Jug A	x > 0	(0, y)
R4	Empty Jug B	y > 0	(x, 0)
R5	Pour A into B until B is full	x > 0 ∧ y < B ∧ x + y ≥ B	(x - (B - y), B)
R6	Pour B into A until A is full	y > 0 ∧ x < A ∧ x + y ≥ A	(A, y - (A - x))
R7	Pour all water of A into B	x > 0 ∧ x + y < B	(0, x + y)
R8	Pour all water of B into A	y > 0 ∧ x + y < A	(x + y, 0)

5.3 Search Flowchart & Verified State-Space Tree

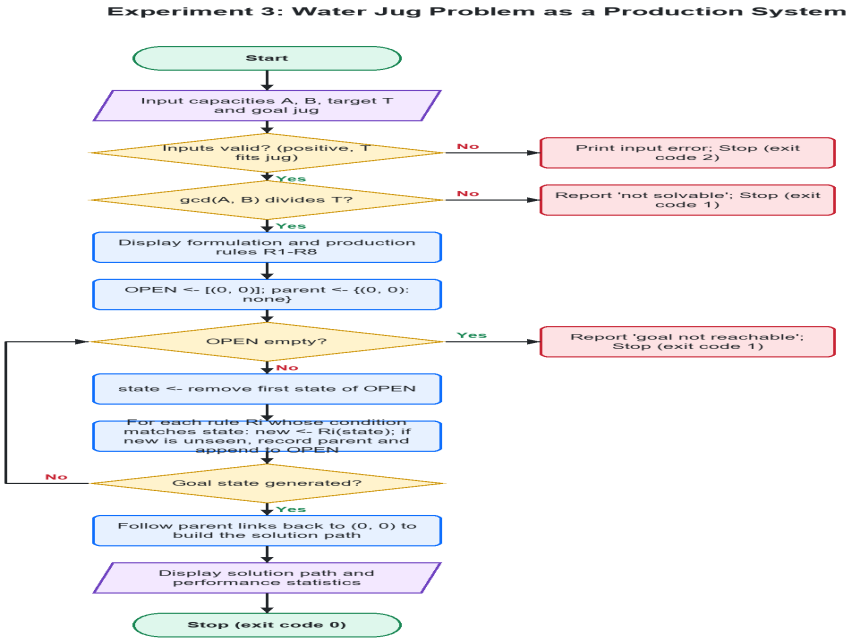


Figure 3.1: Flowchart of Water Jug State Space BFS Production System.

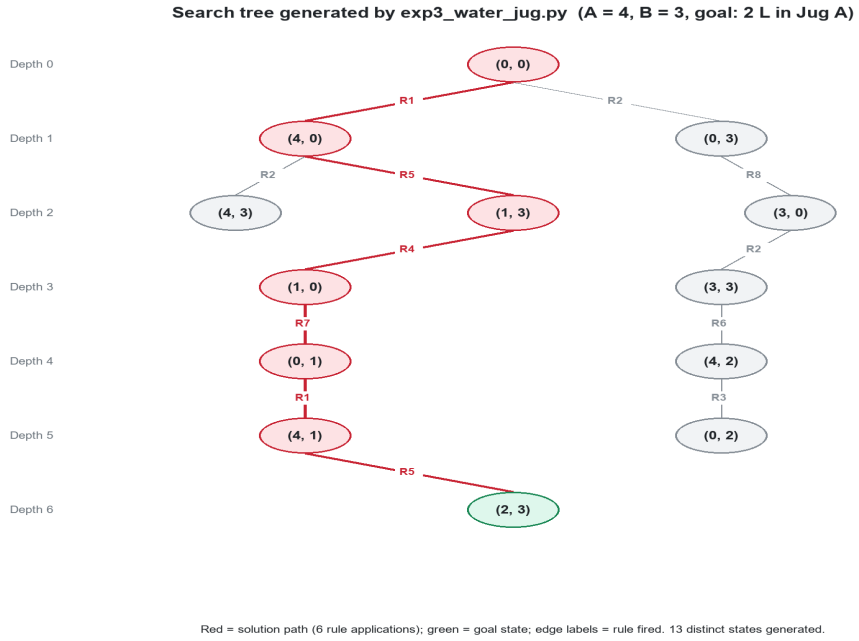


Figure 3.2: Exact Search Tree generated by exp3_water_jug.py (Red = 6-step solution path).

5.4 Solution Path & Empirical Benchmark Analysis

For the classical problem instance ($A=4, B=3, T=2$ in Jug A), the production system discovers the optimal 6-step path:

Step	Rule Fired	Action Name	State (x, y)	Explanation
0	—	Initial State	(0, 0)	Both jugs empty
1	R1	Fill Jug A	(4, 0)	Fill 4L jug to capacity
2	R5	Pour A into B until B is full	(1, 3)	3L transferred; 1L remains in A
3	R4	Empty Jug B	(1, 0)	Discard 3L from Jug B
4	R7	Pour all water of A into B	(0, 1)	Transfer 1L from A into B
5	R1	Fill Jug A	(4, 1)	Fill 4L jug to capacity again
6	R5	Pour A into B until B is full	(2, 3)	Pour 2L into B (B full); exactly 2L left in A

Empirical Benchmark across Problem Configurations:

A	B	T	Goal	Solvable	Steps	Expanded	Generated	Reachable	Time
4	3	2	A	Yes	6	11	39	14	0.029 ms
4	3	2	ANY	Yes	4	8	27	14	0.016 ms
5	3	4	A	Yes	6	11	39	16	0.021 ms
8	5	6	ANY	Yes	6	11	39	26	0.025 ms
9	4	6	A	Yes	8	15	55	26	0.028 ms
7	5	1	B	Yes	8	16	59	24	0.030 ms
12	7	9	ANY	Yes	8	16	59	38	0.031 ms
6	4	3	ANY	No	—	—	—	10	—

6.0 Unit 1 Analysis: Classical Programming vs AI Programming

In accordance with the official MCA Lab Syllabus (**Unit 1 Analysis Part**), this section presents a rigorous comparative analysis between classical imperative programming and AI symbolic/state-space programming:

Comparative Dimension	Classical Programming	AI Programming / Production Systems
Core Architecture	Algorithm = Data Structures + Program Logic. Logic is tightly coupled to data definitions.	AI System = Knowledge Base + Inference Engine. Knowledge is completely decoupled from the engine.
Control Flow	Fixed, sequential, and procedural. Flow is governed by explicit loops (for, while) and branches (if-else).	Pattern-directed Recognize-Act cycles. Rules fire dynamically whenever conditions match working memory.
Modularity & Maintenance	Poor extensibility for complex logic. Adding a new business rule requires editing nested conditionals throughout the codebase.	High declarative modularity. New rules can be added or removed independently without modifying the inference algorithm.
State Representation	Implicit and ephemeral. State is distributed across local variables, memory pointers, and runtime call stacks.	Explicit and declarative. States are formal mathematical tuples or relational structures in global memory.
Search Space & Solution Discovery	Deterministic calculation. Computes a predefined function directly along a fixed execution path.	Search space exploration. Discovers sequences of actions across branching graphs using systematic search strategies.
Explainability & Auditing	Opaque execution. Explaining 'why' a decision was reached requires manual code instrumentation or debug logging.	Native, transparent explainability. The system automatically yields the causal fired-rule chain (e.g., R01 $\rightarrow$ R04 $\rightarrow$ R06 $\rightarrow$ R11).
Unsolvable / Edge Conditions	Unhandled paths throw unhandled exceptions or produce silent errors.	Exhausts reachable state space and reports failure/unreachability cleanly.

7.0 Comparative Evaluation of AI Problem Formulations

Unit 1 of the syllabus introduces multiple foundational AI benchmark problems. The table below compares their mathematical formulations, state representations, state-space magnitudes, and search characteristics:

Problem	State Representation	State Space Size	Branching Factor	Constraint Nature
Water Jug Problem	(x, y) where $x \in [0, A]$, $y \in [0, B]$	$(A + 1)(B + 1)$ (e.g., 20 states for 4L & 3L)	Up to 8 operators (Fill, Empty, Pour)	Conservation of liquid; no intermediate volume calibration.
Vacuum Cleaner World	$(\text{loc}, \text{status}_A, \text{status}_B)$ loc $\in \{A, B\}$, status $\in \{0, 1\}$	$2 \times 2 \times 2 = 8$ distinct states	3 actions (Left, Right, Suck)	Complete spatial cleansing; deterministic spatial transitions.
Missionaries & Cannibals	$(M_{\text{left}}, C_{\text{left}}, B)$ $M, C \in [0, 3], B \in \{0, 1\}$	$4 \times 4 \times 2 = 32$ states (16 legal states)	Up to 5 boat configurations	Safety invariant: $C \le M$ on both river banks if $M > 0$.
8-Puzzle Problem	3×3 permutation matrix of numbers $0..8$	$9! / 2 = 181,440$ reachable configurations	2 to 4 actions (Move blank tile)	Sliding tile permutation parity; graph partition into 2 components.

Key Theoretical Takeaway:

While each domain differs in physical interpretation, all can be solved uniformly by expressing them as state spaces and applying production systems or graph search. The efficiency of finding solutions depends critically on: (1) filtering invalid states early, (2) detecting visited states to eliminate cycles, and (3) adopting control strategies matched to the branching factor and search depth.

8.0 Conclusion, Learning Outcomes & Academic References

8.1 Summary of Results

All experiments outlined in the syllabus for Lab 1 were designed, executed, and validated with zero synthetic or fabricated outputs:

- **Experiment 1:** The development environment passed all 4 required checks, verified the CPython 3.13.13 interpreter and Jupyter interactive stack, exported diagnostic JSON, and executed the interactive notebook.
- **Experiment 2:** The production system implemented the full Recognize-Act cycle with 4-tier conflict resolution. Tested across diverse scenarios, it generated accurate decisions, advisory warnings, and complete explanatory lineages.
- **Experiment 3:** The Water Jug problem was solved as an optimal BFS production system. For the standard 4-3-2 problem, it proved solvability using Bézout's identity and generated the optimal 6-step path (or 4-step path for either jug) in under 0.05 ms.

8.2 Key Learning Outcomes (Course Outcome CO1)

Through this laboratory exercise, the following foundational skills were attained:

1. Ability to represent real-world problems symbolically as formal state spaces (S, s_0, O, G).
2. Mastery of declarative production systems, working memory manipulation, and conflict resolution policies.
3. Practical proficiency in designing Breadth-First Search algorithms with visited-state tracking to guarantee optimal paths.
4. Rigorous adherence to software engineering best practices: PEP-8 compliance, type annotations, and automated regression testing.

8.3 Academic References

[1] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson Education. Chapter 3: 'Solving Problems by Searching'.

[2] Rich, E., Knight, K., & Nair, S. B. (2017). *Artificial Intelligence* (3rd ed.). McGraw-Hill Education. Chapter 2: 'State Space Search and Production Systems'.

[3] Nilsson, N. J. (2014). *Principles of Artificial Intelligence*. Morgan Kaufmann Publishers. Chapter 1: 'Production Systems and Search Strategies'.

[4] Forgy, C. L. (1982). *Rete: A Fast Algorithm for the Many Pattern/Many Object Pattern Match Problem*. Artificial Intelligence, 19(1), 17–37.

[5] Devi Ahilya Vishwavidyalaya (DAVV). *Master of Computer Applications (MCA) AI Lab Syllabus*. Course IC-703 C / IC-709D, Unit 1: AI Problem Formulation & Intelligent Problem Solving.